

Sección 5

APLICACIONES CRIPTOGRAFICAS

§1 Raíces Primitivas

Definición Si p es un número primo y g es un entero no divisible por p , diremos que g es una *raíz primitiva* módulo p si y sólo si $\text{ord}_p(g) = p - 1$.

Por ejemplo, 2 y 6 son raíces primitivas módulo 13, mientras que 3 no lo es, ya que $\text{ord}_{13}(3) = 3$. En cambio, 3 es una raíz primitiva módulo 17. Le dejamos a su cargo la verificación de estas afirmaciones. Claramente, la definición anterior sólo depende de la clase de g módulo p , y es equivalente a afirmar que todo entero no divisible por p es congruente módulo p con una potencia de g . Dicho más simplemente, para cada entero x en el rango $1 \leq x \leq p - 1$ existe un único exponente j ($0 \leq j \leq p - 2$) tal que $x \equiv g^j \pmod{p}$. Debido a este hecho, también diremos que la clase de g , ó simplemente g , es un *generador* del conjunto de clases no nulas módulo p , que notamos $\mathbb{Z}_p^*$.

Enunciaremos sin demostración el siguiente e importante teorema:

5.1.1 Si p es un número primo, existen raíces primitivas módulo p . Más precisamente, existen $\phi(p - 1)$ raíces primitivas módulo p .

El teorema 5.1.5 asegura la existencia de raíces primitivas, pero su demostración no provee ningún método para encontrar una. Si p es relativamente pequeño puede procederse por ensayo y error hasta dar con alguna. Por ejemplo, 2 y 3 no son raíces primitivas módulo 23 ($2^{11} \equiv 3^{11} \equiv 1$), pero 5 sí lo es, como puede verificarse efectuando los cálculos. Habiendo obtenido una podemos fácilmente encontrar todas, ya que ellas son de la forma 5^j , con j variando entre 0 y 21 y coprimo con 22. Siendo $\phi(22) = 10$, resulta que hay 10 elementos de orden máximo en $\mathbb{Z}_{23}^*$, a saber, las clases de 5, 10, 20, 17, 11, 21, 19, 15, 7 y 14, que se obtienen elevando sucesivamente 5 a los exponentes 1, 3, 5, 7, 9, 13, 15, 17, 19 y 21. Por supuesto que los cálculos se efectúan reduciendo siempre módulo 23, y no requieren en este caso mayor esfuerzo, habida cuenta de que $5^2 \equiv 2$.

Si p es grande la situación es bien distinta, y no existe en general ningún método realmente efectivo para hallar una primitiva módulo p . Existen sin embargo un criterio que en algunos casos permite aliviar el problema, como veremos a continuación:

5.1.2 (Criterio) Sea p un número primo, sean $q_1, q_2, \dots, q_m$ los divisores primos de $p - 1$, y supongamos que g es un entero no divisible por p tal que $g^{(p-1)/q_i} \not\equiv 1 \pmod{p}$ para todo i . Entonces g es una raíz primitiva módulo p .

Dem. Supongamos $\text{ord}_p(g) = d$ y que $d < p - 1$. Escribiendo $p - 1 = dk$, resulta que $k > 1$ y por lo tanto es divisible por algún primo. Puesto que por transitividad éste divide a $p - 1$, sigue que existe un índice j y un entero s tal que $k = q_j s$. Tomando congruencias módulo p tenemos entonces que

$$g^{(p-1)/q_j} = g^{ds} = (g^d)^s \equiv 1,$$

lo que contradice las hipótesis. Por lo tanto, $d = p - 1$, esto es, g es una raíz primitiva módulo p .

Aplicaremos 5.1.2 para demostrar que 3 es una raíz primitiva módulo 79. En este caso, $78 = 2 \cdot 3 \cdot 13$, por lo que bastará verificar que ninguno de los números 3^6 , 3^{26} y 3^{39} es congruente con 1 módulo 79. Teniendo en cuenta que $3^4 \equiv 2 \pmod{79}$, obtenemos sucesivamente (todas las congruencias son módulo 79):

- $3^6 = 3^4 \cdot 3 \equiv 18$
- $3^{12} = (3^4)^3 \equiv 8 \Rightarrow 3^{13} \equiv 24 \Rightarrow 3^{26} \equiv 24^2 \equiv 23$
- $3^{39} = 3^{13} 3^{26} \equiv 24 \cdot 23 \equiv 78$

Queda probada entonces nuestra afirmación.

Si bien el criterio anterior reduce sensiblemente la cantidad de pruebas a realizar (recordemos que g es primitiva si y sólo si $g^d \not\equiv 1$ para todo divisor d de $p - 1$ menor que $p - 1$), su aplicación presupone conocer la factorización de $p - 1$, que no es sencilla de obtener si p es muy grande, aún tratándose de un número par. Debemos tener en claro entonces que se trata de un criterio que no es aplicable a cualquier situación.

Observemos también que hemos ilustrado la situación con un ejemplo sencillo (79 no responde a la idea de primo “grande”), y hemos realizado las operaciones empleando ciertas destrezas de cálculo. Para trabajar con primos p de considerable magnitud es necesario desarrollar técnicas específicas de cálculo. Ahorrando algunos detalles, exhibiremos ahora un algoritmo que permite calcular en general potencias módulo n elevando sucesivamente al cuadrado y reduciendo módulo n , y que por lo tanto sólo trabaja con números cuyo tamaño no supera a n^2 .

Un algoritmo para calcular potencias modulares

Supongamos que n es un número natural y que queremos calcular b^m módulo n , donde b es un entero que ya podemos suponer reducido módulo n . La idea es ir elevando sucesivamente al cuadrado, efectuando en cada paso las correspondientes reducciones. El algoritmo comporta una serie de etapas, cada una de las cuales presenta una disyuntiva. Le mostraremos cómo empieza y cuál es en cada caso la disyuntiva que se presenta, señalando entonces cuál es el próximo paso a realizar. Tenemos de entrada dos posibilidades, que luego se repetirán en cada paso (todas las operaciones indicadas se realizan módulo n):

- (i) m es par. Podemos escribir entonces $b^m = (b^2)^{m/2}$. Para continuar, reemplazamos b por b^2 , el exponente m por $m/2$, y procedemos de igual forma, ahora con respecto al nuevo exponente.
- (ii) m es impar. En tal caso escribimos $b^m = b(b^2)^{(m-1)/2}$. Nuevamente reemplazamos b por b^2 y m por $(m-1)/2$, sólo que ahora tenemos un factor b que debemos acumular. Continuamos luego como en el caso (i).

Estas son entonces las sucesivas etapas del algoritmo, que finaliza cuando el exponente alcanza el valor 1. Puesto que cada vez dividimos por 2 el exponente (más precisamente, tomamos la parte entera de su mitad), el número de etapas coincide con el número de cifras del desarrollo binario de m . En cada una de ellas deberemos elevar un número menor que n al cuadrado, por lo que los números involucrados no superarán a n^2 . A su vez, en algunas de las etapas deberemos multiplicar por el valor acumulado (en aquellas en las que el exponente es impar), coincidiendo el número de las mismas con la cantidad de bits no nulos del desarrollo binario de m . Puesto que es relativamente sencillo demostrar que el valor esperado de esta cantidad es aproximadamente la mitad de sus cifras binarias, concluimos que el número de multiplicaciones que requerirá el algoritmo será del orden de $1.5 \times \log_2 m$, lo que lo hace muy eficiente.

Vamos a desarrollar completamente un ejemplo, empleando 3 variables a , k y c , cuyos roles son los siguientes:

- La variable a toma el valor inicial b y será reemplazada en cada paso por $a^2 \bmod n$.
- La variable k indica el exponente, cuyo valor inicial es m . La reemplazaremos en cada etapa por la parte entera de su mitad.
- La variable c indica el valor acumulado, cuyo valor inicial es 1. En cada etapa, permanece inalterable si el exponente es par y se la multiplica por a si es impar. Su valor final es el resultado buscado.

Todos estos datos serán dispuestos en una tabla, como mostramos a continuación en el siguiente ejemplo (el orden de modificación de los parámetros es $c - a - k$):

Ejemplo Tomemos $n = 113$, $b = 37$ y $m = 76$ (le encargamos que verifique todos los resultados parciales).

Etapas	a	k	c
0	37	76	1
1	13	38	1
2	56	19	1
3	85	9	56
4	106	4	14
5	49	2	14
6	28	1	14
7			53

Hemos obtenido entonces que $37^{76} \equiv 53 \pmod{113}$. Obsérvese que hemos alterado a c (multiplicándolo por a) en las etapas 3, 4 y 7, que corresponden a valores impares del exponente en las etapas previas 2, 3 y 6.

§2 Primalidad y Factorización

A lo largo de estas páginas nos hemos referido ya en varias oportunidades a la complejidad computacional que presenta la determinación de la primalidad de un número natural muy grande, así como a la dificultad de factorizarlo en el caso de que no sea primo. En este apartado mencionaremos brevemente algunos tipos de técnicas y estrategias que se emplean para resolver estos problemas, por cierto muy sucintamente, dado que dichos métodos emplean herramientas que están fuera del alcance de estas notas.

Las dos cuestiones anteriores —obviamente relacionadas—, tienen sin embargo disímiles grados de dificultad. Se verá luego cómo el uso de la teoría permite decidir con bastante eficacia y razonable certeza (ya precisaremos qué significa esto) si un número es primo, mientras que es considerablemente más arduo el problema de encontrar algún factor no trivial de un número que ha sido establecido previamente como compuesto. Ambos problemas son de rigurosa actualidad, ya que los modernos sistemas criptográficos —que más adelante expondremos— requieren el empleo de números primos muy grandes y basan su seguridad en la imposibilidad práctica de factorizar números compuestos también de gran tamaño.

En cuanto a la primera cuestión, hasta aquí hemos mencionado dos posibles formas de determinar la primalidad de un número natural n . El método de la criba de Eratóstenes, que consiste en verificar que n no es divisible por ningún primo menor ó igual que su raíz cuadrada, y el método de factorización de Fermat.

Podemos mencionar otro, basado en el teorema de Wilson, que afirma que un número natural n es primo si y sólo si $(n-1)! \equiv -1 \pmod{n}$ (su demostración es consecuencia directa del hecho de que todo entero no divisible por un cierto primo p admite inverso modular módulo p). Los tres constituyen *tests determinísticos* de primalidad, en el sentido de que aseguran completamente el carácter primo de n si éste supera la prueba. Desafortunadamente —como también ya hemos comentado—, ninguno de ellos es eficiente si n es muy grande, debido a la enorme cantidad de ensayos y cálculos que requieren.

Existen sin embargo otros tipos de tests, los *tests probabilísticos* de primalidad, que someten a un número a varias rondas de prueba, y que pueden ser aplicados a números de gran porte por ser mucho más factibles desde el punto de vista computacional que los tests determinísticos. Según sugiere su nombre, ellos no afirman rotundamente que un número que sortea dichas pruebas sea primo, sino que brindan más bien una razonable certeza de que lo sea, con una probabilidad que además puede estimarse. Adicionalmente, todos ellos comparten una importante característica: si un número no pasa el test entonces es compuesto sin lugar a dudas.

Describiremos a continuación uno de los tests probabilísticos de primalidad más conocidos. Como veremos, la idea básica consiste en someter al número en cuestión a una serie de verificaciones, que debieran arrojar un resultado positivo si el número fuese primo.

El Test de Fermat

Dado un número natural n —obviamente impar— cuya primalidad quiere estudiarse, elegimos al azar un número a en el rango $1 < a < n-1$ y calculamos $x = a^{n-1} \pmod{n}$. Si $x \neq 1$, el teorema de Fermat asegura que n es compuesto y el test finaliza. Si $x = 1$ nada podemos asegurar, pero podemos empezar a sospechar tenuemente que n es primo. Diremos en tal caso que n *pasa el test* respecto a la base a , ó que n es un *probable primo* respecto a la base a . Continuamos luego el test eligiendo otras bases y realizando cada vez la misma prueba. Si para alguna base b resulta que $b^{n-1} \not\equiv 1 \pmod{n}$ el test finaliza con la conclusión de que n es compuesto. En caso contrario, cada ronda aumentará nuestra sospecha acerca de la primalidad de n .

Nota Como precaución adicional, cada vez que se elige una base b es conveniente calcular previamente $(b : n)$ mediante el algoritmo de Euclides. Si

resulta distinto de 1 habremos encontrado un factor propio de n , resultando que n es compuesto. Por lo tanto, siempre aplicaremos el test respecto a bases coprimas con n . Observemos además que nunca elegimos ni a 1 ni a $n - 1$ (que es congruente con -1 módulo n) como bases, ya que claramente n pasará el test respecto a ellas.

Hemos hasta aquí descrito el método, pero es claro que queda un gran interrogante: ¿Con cuántas bases debemos probar para estar suficientemente convencidos de que n es primo? El siguiente lema ayudará a dilucidar la cuestión:

5.2.1 Supongamos que n no pasa el test con respecto a una cierta base b . Entonces n es un probable primo respecto a lo sumo el 50 % de las bases coprimas con n .

Siendo b coprimo con n , recordemos que la aplicación $a \mapsto ab$ módulo n determina una permutación de las bases coprimas con n . Si n es un probable primo respecto a la base a , resulta que $(ab)^{n-1} = a^{n-1}b^{n-1} \equiv b^{n-1} \not\equiv 1 \pmod{n}$, por hipótesis. Por lo tanto, a cada base respecto a la cual n pasa el test le corresponde otra respecto a la cual no lo pasa. Puesto que la correspondencia es inyectiva, concluimos que el número de las primeras es menor ó igual que el de las últimas, lo que claramente demuestra el lema.

Tomando por ejemplo $n = 133 = 7 \cdot 19$ (es obviamente un número muy pequeño, pero sólo queremos ilustrar la situación) y realizando los correspondientes cálculos, resulta que 133 es un probable primo respecto a 36 bases coprimas, aproximadamente el 33 % de las $\phi(133) = 108$ bases coprimas con 133.

Lo invitamos a resolver la misma cuestión para $n = 91$, usando métodos similares. Podrá comprobar que el número de bases respecto a las cuales 91 pasa el test es exactamente la mitad de las posibles bases, lo que indica que la cota del lema es óptima.

El lema 5.2.1 nos muestra que si n es compuesto y no pasa el test para todas las posibles bases entonces tendremos por lo menos el 50 % de chance de que no lo pase respecto a una base elegida al azar. En otras palabras, si n pasa el test respecto a una cierta base, *existe una probabilidad menor ó igual que 1/2* de que n no sea primo. Si lo hace con respecto a dos bases dicha probabilidad será menor ó igual que 1/4 y, en general, la probabilidad de que n sea compuesto y pase el test de Fermat respecto a k bases es menor ó igual que $1/2^k$. A partir de este dato cuantitativo podemos estimar el número de rondas que ejecutaremos del test antes de declarar primo a n , dependiendo del grado de certidumbre que querramos tener. Por ejemplo, si deseamos tener

un grado de confianza superior al 99 % respecto a la primalidad de n , éste deberá sortear por lo menos 7 rondas del test, ya que $2^{-7} = 0.0078125 < 0.01$.

Ejemplo 5.2.2 Compruebe que 45 es un probable primo respecto a ocho bases coprimas con 45. Determine tales bases.

Pseudoprimos La implementación del test de Fermat que hemos descripto en el párrafo anterior descansa en la conclusión del lema 5.2.1 —sobre la que se apoya nuestro análisis de la probabilidad— bajo la hipótesis de que n no pasa el test respecto a *por lo menos una base*. Debemos hacer sin embargo una importante salvedad, que descubrirá una pequeñísima debilidad del test. En efecto, existe una clase muy especial de números impares compuestos que pasan el test respecto a *todas las bases*, es decir, números n con la propiedad de que $a^{n-1} \equiv 1 \pmod{n}$ para todo a coprimo con n . Son los llamados *pseudoprimos* ó números de Carmichael, en homenaje al matemático inglés R. D. Carmichael, que los introdujo en 1912. Por ejemplo, $561 = 3 \times 11 \times 17$ es pseudoprimo (de hecho es el menor de ellos), ya que si a es coprimo con 3, 11 y 17, sigue aplicando sucesivamente el teorema de Fermat que:

$$a^{560} = (a^2)^{280} \equiv 1 \pmod{3} \quad (3)$$

$$a^{560} = (a^{10})^{56} \equiv 1 \pmod{11} \quad (11)$$

$$a^{560} = (a^{16})^{35} \equiv 1 \pmod{17}, \quad (17),$$

resultando que $a^{560} \equiv 1 \pmod{561}$.

El ejemplo que acabamos de mostrar sugiere con bastante claridad qué tipo de factorización debe tener un número pseudoprimo. Concretamente, la siguiente proposición —que enunciaremos sin demostración— los caracteriza completamente:

5.2.3 Un número impar n es pseudoprimo si y sólo si su factorización es de la forma

$$n = \prod_{i=1}^r p_i,$$

donde $r \geq 3$, los p_i son primos distintos y $p_i - 1 \mid n - 1$ para todo i .

Por ejemplo, $1105 = 5 \times 13 \times 17$ y $1729 = 7 \times 13 \times 19$ verifican las condiciones anteriores, y son por lo tanto números de Carmichael.

Los números de Carmichael son realmente raros —las condiciones de 5.2.3 son muy restrictivas— por lo que su existencia no invalida la utilidad del test de Fermat. Digamos por ejemplo que sólo hay 2163 pseudoprimos menores que 25×10^9 , algo así como ¡ uno cada 11 millones y medio de números ! Paradójicamente, Alford, Granville y Pomerance demostraron en 1992 que

existen infinitos números de Carmichael, un notable resultado de muy compleja demostración.

Nota Existen otros tests de primalidad, como el test de Solovay-Strassen, también conocido como test de Jacobi, y el test de Rabin-Miller. Ambos son refinamientos del test de Fermat, operan en forma similar a éste, pero son algo más eficientes. De hecho, puede probarse que si un número natural n supera cualquiera de estos dos nuevos tests respecto a una base a entonces también supera el test de Fermat respecto de a , pero la reciproca no es cierta. Así, es posible que n pase el test de Fermat pero no alguno de los otros dos. En realidad, el de Rabin-Miller es el más fuerte de los tres (por ello suele mencionarse como el *test de primalidad fuerte*). En su caso, resulta que si n pasa el test respecto de una cierta base existe una probabilidad menor o igual que $1/4$ de que n no sea primo. Esto significa, por ejemplo, que si n pasa 20 rondas del test de Rabin-Miller existirá una chance de a lo sumo ¡uno en un billón! de que no sea primo.

Como último apunte respecto a los tests de primalidad, es indudable que sólo hemos hecho una descripción sumaria de los mismos, sin ahondar en todos los detalles teóricos y de implementación. Hemos obviado en particular un aspecto crucial, como es el análisis de la complejidad de los algoritmos diseñados para su funcionamiento, absolutamente necesario para poder garantizar su factibilidad. En cuanto a los ejemplos desarrollados, es obvio que son algo ingenuos, pero los hemos ofrecido con la expresa intención de ilustrar los diversos métodos descriptos, ya que como hemos señalado, los tests anteriores —sobre todo el de Rabin-Miller— revelan su potencia y su eficacia cuando se los aplica a números de gran tamaño. Digamos de paso que la estimación teórica de la probabilidad que hemos efectuado sobre el test de primalidad fuerte parece ser bastante conservadora, ya que la práctica apunta a señalar otra cosa. Por ejemplo, se ha determinado que sólo un número impar compuesto menor que $2,5 \times 10^{10}$ (explícitamente 3215031751) supera el TFP respecto a las bases 2, 3, 5 y 7. Mencionemos finalmente que está planteada la siguiente conjetura, que de poder ser probada reduciría enormemente la cantidad de pruebas a efectuar: si n es compuesto, entonces no pasa el test de primalidad fuerte respecto a alguna base menor que $2 \times \log^2 n$ (log denota logaritmo natural).

Factorización En cuanto a factorizar, recordemos que hemos desarrollado ya el método de factorización de Fermat, razonablemente eficiente sólo si el número a factorizar admite algún divisor cercano a su raíz cuadrada. Si bien no describiremos explícitamente aquí ningún otro, haremos una breve referencia a una técnica ideada originalmente por Maurice Kraitchik, que

resulta ser una extensión del método de Fermat.

Dicha generalización es la siguiente: dado un número n que sabemos compuesto, en vez de expresar n como diferencia de cuadrados lo que hacemos es buscar dos cuadrados que *difieran en un múltiplo de n* . Si bien esta condición es más débil y no brindará directamente una factorización de n , es menos costoso desde el punto de vista computacional hallar un par de tales números, y una vez hallados tendremos una buena chance de obtener un factor propio de n .

Para justificar la última afirmación, supongamos que x e y son números naturales satisfaciendo la relación $x^2 \equiv y^2 \pmod{n}$ y tales que $y \not\equiv \pm x \pmod{n}$. Equivalentemente, $n \mid x^2 - y^2$ pero $n \nmid x - y$ y $n \nmid x + y$. Esto es imposible que ocurra si n es primo, pero perfectamente puede ocurrir si n es compuesto, y significaría informalmente hablando que los divisores primos de n están “repartidos” entre los dos factores $x - y$ y $x + y$. Sin entrar en detalles, señalemos que la chance de que ello efectivamente ocurra es de alrededor del 50 %. En tal caso, $d = (x - y : n)$ será un factor propio de n . En efecto, si $d = 1$ resultaría que $n \mid x + y$, mientras que el caso $d = n$ tampoco es posible, ya que $n \nmid x - y$.

Enfatizemos que el éxito del método depende de la validez de las condiciones $y \not\equiv \pm x \pmod{n}$. Si estas no se cumplieran habrá que intentar con otros pares, hasta dar con uno que las satisfaga.

Habiendo ya expuesto la idea básica del método y su razonabilidad, nos ocuparemos ahora de un hecho crucial: ¿cómo obtener pares de enteros (x, y) tales que $x^2 \equiv y^2 \pmod{n}$? Aquí las técnicas se separan —existen distintas formas de hacerlo—, y la utilización de una u otra depende del tamaño del número a factorizar. Sólo agregaremos aquí que todas ellas requieren conocimientos más avanzados de Matemática y en especial de teoría de números. Por ejemplo el método de la *criba cuadrática*, creado por Pomerance a fines de los 80, que usa resultados de la teoría de residuos cuadráticos.

Como todos los métodos de factorización, el que acabamos de describir tiene su cuota de ensayo y error. Si la diferencia de cuadrados divisible por n que hallamos no nos brinda un factor propio de n deberemos intentar con otra, que podemos hallar empleando algunas de las técnicas mencionadas arriba. Debe quedar claro sin embargo que el algoritmo es probabilístico pero no azaroso. Sin entrar en detalles, decimos esto no solamente porque realmente funciona en la práctica, sino porque la elección de los diversos parámetros utilizados para fabricar los pares (x, y) está cuidadosamente sustentada en hechos teóricos que la hacen eficaz.

§3 Aplicaciones en Criptografía

Desde tiempos muy remotos el hombre ha utilizado diversas artimañas para resguardar la privacidad de ciertos mensajes, transformándolos de forma que resultasen incomprensibles para cualquier tercero que lograra interceptarlos, pero de manera que pudieran ser descifrados por el destinatario mediante una fórmula previamente acordada. El asunto ha tenido un extraordinario impulso a través de la Historia, y lo que empezó siendo casi un juego de ingenio se ha convertido actualmente en una compleja disciplina de estudio, *la criptografía*. La palabra es de origen griego —conjunción de *kriptós* (oculto) y *grapho* (escribir)—, y designa tanto el estudio de las numerosas técnicas ideadas para enmascarar un texto como los diversos análisis que se practican sobre los mensajes codificados para desentrañarlos (*criptoanálisis*). Si bien sólo brindaremos un breve apunte sobre el tema, resultará interesante mostrar el uso de la Aritmética en algunos sistemas criptográficos actuales.

Comenzaremos enumerando algunos términos de uso común en la disciplina. Supondremos por razones de simplicidad que trabajamos con los 27 caracteres del idioma castellano (el alfabeto). Por supuesto que en la práctica se agregan otros, como signos de puntuación, espacios, dígitos, etc.

- **Texto claro** Designa el mensaje real a enviar por el emisor.
- **Cifrado** Se refiere al proceso de transformación practicado sobre el texto claro para ocultarlo.
- **Texto cifrado** Es la secuencia de caracteres que resulta de cifrar el texto claro, y que se envía al receptor.
- **Descifrado** Es el proceso de transformación (inverso al cifrado) que se efectúa sobre el texto cifrado para recuperar el mensaje.
- **Sistema criptográfico** Se refiere al tipo de transformación que se aplicará al texto claro para obtener el cifrado. A cada transformación deberá corresponder una transformación inversa, que aplicará el receptor para descifrar el mensaje.
- **Clave** Designa la transformación específica que se utilizará, previamente acordada entre el emisor y el receptor. Puesto que en la práctica es frecuente que el mensaje pueda ser interceptado por alguien que tenga conocimiento del sistema criptográfico que se está empleando, la seguridad del sistema depende del mantenimiento en secreto de la clave.

Veamos algunos ejemplos:

- (1) El sistema consiste en transformar las letras del texto claro aplicando cualquiera de las $27!$ permutaciones del alfabeto. La permutación particular que se emplee será en este caso la clave. Por ejemplo, si usamos la permutación

A	B	C	D	E	F	G	H	I	J	K	L	M	N	Ñ	O	P	Q	R	S	T	U	V	W	X	Y	Z
K	R	Z	H	O	W	E	M	T	B	J	Q	Y	G	Ñ	V	D	L	S	A	I	P	X	F	N	U	C

el texto claro *SECRETO* se transforma en el texto cifrado *AOZSOIV*. La segunda fila del esquema anterior se llamará una *tabla de alfabeto*, y la operación inversa de descifrado se obtiene simplemente reemplazando cada letra del cifrado por la que está arriba de ella en la tabla de alfabeto correspondiente. Es evidente que este sistema permite disponer de una fantástica cantidad de claves que torna inútil el criptoanálisis por ensayo y error, pero también es cierto que el intercambio de claves entre el emisor y el receptor podría presentar alguna dificultad, habida cuenta de que la inmensa mayoría de las transformaciones no responde a ninguna regla sencilla de comunicar.

- (2) Una manera de obviar esta última dificultad sería restringirnos a un grupo más pequeño de transformaciones, de descripción más simple. Un ejemplo histórico de tales permutaciones son las llamadas *transformaciones César*, que como indica su nombre se remontan a la época del Imperio Romano. Las mismas consisten en desplazar uniforme y cíclicamente cada letra del alfabeto un cierto número de lugares. Por ejemplo, si desplazamos 12 posiciones cada letra, obtenemos la siguiente tabla de alfabeto, ó tabla de desplazamiento César, que comenzará con *M*:

A	B	C	D	E	F	G	H	I	J	K	L	M	N	Ñ	O	P	Q	R	S	T	U	V	W	X	Y	Z
M	N	Ñ	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L

Bajo esta transformación, *SECRETO* se convierte en *EPÑDPFA*. Notemos que hay 27 transformaciones César, y que cada clave puede rotularse simplemente por un número entero entre 0 y 26, que representa la cantidad de posiciones que se desplaza cada letra. Más aún, la transformación responde a una fórmula aritmética sencilla. En efecto, si numeramos las letras como sigue:

A	B	C	D	E	F	G	H	I	J	K	L	M	N	Ñ	O	P	Q	R	S	T	U	V	W	X	Y	Z
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26

es inmediato verificar que la transformada de cada letra se obtiene sumando 12 al número que le corresponde y reduciendo luego, si es

necesario, módulo 27. Así, a la G (6) le corresponde la R (18), mientras que a la V (22) le corresponde la H (7), ya que $22+12 = 7 \bmod 27$. Naturalmente, el descifrado se obtiene aplicando la transformación inversa de la anterior, esto es, restando 12 módulo 27. Es interesante hacer notar también que el hecho de poder describir la transformación mediante una fórmula matemática permite automatizar más fácilmente las operaciones de cifrado y descifrado, mediante la programación de algoritmos que podrán ser luego ejecutados por una computadora.

Hemos brindado el ejemplo de las transformaciones de César como mera ilustración histórica, ya que en realidad el sistema es extremadamente inseguro. Ello se debe principalmente al siguiente hecho: basta conocer una sola equivalencia entre letras para determinar completamente la clave. En efecto, supongamos que sabemos que a una cierta letra codificada por el número x le corresponde otra cuya codificación es y . En tal caso, la clave desconocida k puede despejarse simplemente de la ecuación $x + k = y \bmod 27$, lo que ya nos pone en condiciones de descifrar el texto entero. Si éste es suficientemente extenso, el criptoanalista podrá obtener la referida equivalencia apelando al llamado *análisis de frecuencia*, que muy brevemente explicado consiste en determinar la letra ó las letras que más aparecen en el texto cifrado, pues muy probablemente corresponderán a las letras con mayor frecuencia estadística del idioma en que fue redactado el texto claro. En un texto demasiado corto quizás no se manifieste esta preminencia estadística, pero en ese caso podrá proceder por simple ensayo y error, habida cuenta de que sólo tiene que probar con 27 claves.

- (3) Observemos que cuando se cifra un texto a través de una única tabla de alfabeto (*sustitución monoalfabética*), lo que posibilita el análisis de frecuencia es el hecho de que a cada letra le corresponde siempre la misma, no importa el lugar que ocupe en el texto. En el siglo XVI, un diplomático francés llamado Blaise de Vigenère ideó un sistema para paliar esta debilidad, que se conoce con el nombre de tabla Vigenère. El mismo es un sistema de *sustitución polialfabética* —ya que utiliza simultáneamente varias tablas de alfabeto—, y responde a la siguiente idea: se elige previamente una palabra ó frase (que constituirá la clave) y se la coloca —repitiéndola tantas veces como haga falta— arriba del texto claro. Para cifrar cada letra de éste, se observa qué letra de la clave tiene por encima, y se le aplica la transformación de César correspondiente al número de dicha letra. Así, si queremos cifrar la F , y ésta tiene por encima la J , buscamos en la tabla de desplazamiento César que comienza con J la letra correspondiente a la F , que resulta

ser la O . A manera de ejemplo, supongamos que el texto claro comienza diciendo *SUPER SECRETO* y que la palabra clave elegida es “*clave*”. El cifrado será entonces:

$$\begin{array}{cccccccccccccccc} c & l & a & v & e & c & l & a & v & e & c & l & \\ S & U & P & E & R & S & E & C & R & E & T & O & \\ U & F & P & Z & V & U & O & C & N & I & V & Z & \end{array}$$

Observemos un par de diferencias esenciales respecto a las sustituciones monoalfabéticas. Una misma letra, la E en este caso, se transforma sucesivamente en Z , O e I , mientras que por otro lado, en el texto cifrado, la V corresponde una vez a la R y otra vez a la T . Es claro que estas características dificultan mucho más la ruptura de estos códigos polialfabéticos, pero apresurémonos a decir —sin entrar en detalles— que los mismos están lejos de ser invulnerables, y que pueden atacarse por ejemplo mediante un análisis de frecuencias más fino que el anterior, ideado por el inglés Babbage en 1854.

Clave pública

La criptografía tiene una larga historia, y los ejemplos elementales de cifrado que hemos brindado sólo han tenido el propósito de ilustrar el tema. Los métodos que hemos descripto fueron sucesivamente reemplazados a lo largo del tiempo por técnicas cada vez más sofisticadas y aparentemente invulnerables de cifrado, hasta que la tenaz e inteligente labor de los criptoanalistas revelaba sus debilidades y obligaba a cambiarlas por otras más seguras. Esta suerte de carrera entre dos bandos —los creadores de códigos y los que intentan romperlos— no ha terminado aún, y la creciente aplicación de la Matemática en la criptografía y el uso de la electrónica la tornan cada vez más apasionante y compleja. El lector interesado en conocer con más detalle algunos de estos avances puede consultar la obra de divulgación “Los códigos secretos”, de Simon Singh [16], que describe con un estilo muy ameno el desarrollo histórico de la criptografía.

Con el advenimiento de la informática, la criptografía dejó de ser un asunto casi siempre reservado a las esferas militares y diplomáticas para convertirse en un tema de interés general. El auge del intercambio digital —correo electrónico, Internet, etc.— y de las operaciones comerciales electrónicas acrecentaron la necesidad de resguardar la privacidad de las comunicaciones, y esto llevó al desarrollo de nuevos métodos criptográficos. Nos vamos a referir en primer término a un problema que constituía en sí mismo un círculo vicioso, como es el de la distribución de claves. En efecto, si dos personas quieren intercambiar un mensaje secreto, el emisor debe primero

codificarlo. Para ello necesita usar una clave, que deberá comunicar previamente al receptor. Como ésta puede ser interceptada, debería a su vez ser codificada, para lo cual también necesitaría una clave, y así *ad infinitum*. Para salvar este inconveniente surgió la ingeniosa idea de clave pública, que no requiere un intercambio previo y explícito de la clave entre los correspondientes. Veremos de paso cómo comienza a entrar en juego la Aritmética.

Intercambio de claves La idea de utilizar una clave pública para intercambiar claves fue introducida en 1976 por dos especialistas norteamericanos en criptografía, W. Diffie y M. Hellman, que idearon un sistema —basado en ciertas funciones de la aritmética modular— que permite que dos correspondientes acuerden una clave *sin necesidad de que ésta sea transmitida*. Veamos en qué consiste.

Supongamos que un correspondiente $\mathcal{X}$ desea acordar una clave k con otro correspondiente $\mathcal{Y}$, que luego utilizarán para cifrar mensajes con algún sistema criptográfico. Comienzan poniéndose de acuerdo en la elección de un número primo p y de una raíz primitiva g módulo p , que no mantendrán en secreto y pueden ser conocidos por cualquier persona. De esta manera, el par (p, g) constituye la *clave pública* del sistema. Posteriormente, $\mathcal{X}$ elige un número x ($1 < x < p - 1$) —que mantiene en secreto—, y lo mismo hace $\mathcal{Y}$, eligiendo un número y en el mismo rango. Estos números serán las respectivas *claves privadas* de los correspondientes, y ninguno de los dos transmitirá la suya al otro. Ahora están listos para acordar la clave k , a través de las siguientes etapas (todas las operaciones se realizan módulo p):

- Etapa 1. $\mathcal{X}$ calcula $u = g^x$ y envía u a $\mathcal{Y}$.
- Etapa 2. $\mathcal{Y}$ calcula $v = g^y$ y envía v a $\mathcal{X}$.
- Etapa 3. $\mathcal{X}$ calcula v^x .
- Etapa 4. $\mathcal{Y}$ calcula u^y .

La clave Los valores de las etapas 3 y 4 coinciden, ya que

$$v^x = (g^y)^x = (g^x)^y = u^y.$$

Pues bien, este resultado común es la clave acordada k .

Por ejemplo, supongamos que $p = 79$, $g = 3$, y que $\mathcal{X}$ e $\mathcal{Y}$ han elegido a 19 y a 28 como sus respectivas claves privadas (desde ya que el ejemplo no es real, se trabaja con números muchísimo más grandes). En ese caso, la clave será

$$k = (3^{19})^{28} \equiv 37^{28} \equiv 45 \pmod{79},$$

como Ud. puede verificar.

Como se ve, han establecido una clave sin necesidad de transmitirla en ningún momento. Ahora bien, ¿qué ocurre si un intruso intercepta alguna ó las dos comunicaciones de las etapas 1 y 2? Puesto que estamos suponiendo que está en posesión de u ó de v (quizás de ambos), todo lo que necesita entonces para descubrir la clave k es conocer x ó y , valores que podría intentar determinar a partir de cualquiera de las ecuaciones $u = g^x$ ó $v = g^y$. Afortunadamente para la seguridad del sistema —en este hecho reside exactamente su integridad—, es muy poco probable que lo logre, ya que si p es suficientemente grande, *no se conoce ningún algoritmo eficiente que permita despejar el exponente en las ecuaciones anteriores.*

Logaritmo discreto En general, dado un número a coprimo con p , se llama *logaritmo discreto* de a en base g al único exponente e ($0 \leq e < p - 1$) que verifica la relación $g^e \equiv a \pmod{p}$. Empleando la notación $e = \log_g(a)$, resulta entonces que el mismo está caracterizado por la condición

$$g^{\log_g(a)} \equiv a \pmod{p}.$$

Debe por supuesto su nombre al obvio parecido formal que presenta su definición respecto a la función logaritmo clásica, y reiteremos que es en la dificultad de calcularlo donde reside la seguridad del método de intercambio de claves que hemos descripto arriba. Resulta así que la función $z \mapsto g^z$ es un primer ejemplo de lo que se da en llamar *función de una sola vía*, esto es, una función que es sencilla de evaluar pero cuya inversa (el logaritmo discreto en este caso) es muy difícil de calcular. Otro ejemplo de proceso de una sola vía es el de multiplicar dos números —sin duda sencillo—, pero cuyo proceso inverso (la factorización) ya vimos puede ser muy complicado. Por sus características, es claro que este tipo de función es especialmente apto para su empleo en la criptografía de clave pública, esto es, en aquellos criptosistemas que realizan sus operaciones de cifrado y descifrado a través de una combinación de claves públicas y secretas. Ello quedará evidenciado en el sistema criptográfico RSA que explicaremos a continuación, en lo que será la última sección de este capítulo.

El sistema RSA

El criptosistema RSA es actualmente uno de los más populares sistemas de clave pública, y debe su denominación a quienes lo crearon en 1978, Rivest, Shamir y Adleman. Su seguridad descansa en la tremenda dificultad de factorizar números de gran tamaño, y su funcionamiento está sustentado en el teorema de Fermat—Euler, que establecimos y demostramos en el capítulo 6. Pasemos a describirlo.

Para fabricar sus claves, cada usuario $\mathcal{X}$ del sistema comienza eligiendo —a través de la aplicación de algún test de primalidad— dos números primos $p_{\mathcal{X}}$ y $q_{\mathcal{X}}$ extremadamente grandes, de unos 150 dígitos cada uno. Los guarda para sí, y hace público su producto $n_{\mathcal{X}} = p_{\mathcal{X}}q_{\mathcal{X}}$. Luego elige al azar un número $e_{\mathcal{X}}$ coprimo con $\phi(n_{\mathcal{X}})$ en el rango $1 < e_{\mathcal{X}} < \phi(n_{\mathcal{X}})$, que también hace público, y procede a calcular mediante el algoritmo de Euclides su inverso módulo $\phi(n_{\mathcal{X}})$. Este número, que designaremos por $d_{\mathcal{X}}$, será mantenido en secreto.

Resumiendo, cada usuario $\mathcal{X}$ dispone de una clave pública y de una clave privada, según el siguiente detalle:

$$\begin{aligned}\text{Clave pública de } \mathcal{X}: & (n_{\mathcal{X}}, e_{\mathcal{X}}) \\ \text{Clave privada de } \mathcal{X}: & (p_{\mathcal{X}}, q_{\mathcal{X}}, d_{\mathcal{X}}),\end{aligned}$$

donde $n_{\mathcal{X}} = p_{\mathcal{X}}q_{\mathcal{X}}$ y $e_{\mathcal{X}}d_{\mathcal{X}} \equiv 1 \pmod{\phi(n_{\mathcal{X}})}$. Notemos que esta última relación es equivalente a la existencia de un entero t tal que $e_{\mathcal{X}}d_{\mathcal{X}} = t\phi(n_{\mathcal{X}}) + 1$.

Observación Alguien podría intentar descubrir el parámetro secreto $d_{\mathcal{X}}$ determinando previamente $\phi(n_{\mathcal{X}})$. Pero en ese caso también sería capaz de determinar $p_{\mathcal{X}}$ y $q_{\mathcal{X}}$, ya que usando la fórmula de cálculo de ϕ sigue inmediatamente que valen las relaciones:

$$\begin{aligned}p_{\mathcal{X}} + q_{\mathcal{X}} &= n_{\mathcal{X}} - \phi(n_{\mathcal{X}}) + 1 \\ p_{\mathcal{X}}q_{\mathcal{X}} &= n_{\mathcal{X}},\end{aligned}$$

resultando que $p_{\mathcal{X}}$ y $q_{\mathcal{X}}$ son las raíces del polinomio cuadrático

$$X^2 - (n_{\mathcal{X}} - \phi(n_{\mathcal{X}}) + 1)X + n_{\mathcal{X}},$$

que obviamente pueden calcularse. Por lo tanto, determinar $\phi(n_{\mathcal{X}})$ es equivalente a obtener la factorización de $n_{\mathcal{X}}$, y ya hemos comentado que no se conoce ningún algoritmo eficaz para factorizar números de tal envergadura.

Veamos cómo opera el sistema RSA. Supongamos que un usuario $\mathcal{Y}$ quiere mandar un mensaje codificado al usuario $\mathcal{X}$, cuya clave pública puede consultar en un directorio. Para simplificar la notación suprimiremos los subíndices, llamando simplemente p , q , n , e y d a los parámetros públicos y privados de $\mathcal{X}$. Los pasos son los siguientes:

- (1) Primero $\mathcal{Y}$ convierte el mensaje en un número natural a menor que n (ya veremos cómo puede implementarse esto).
- (2) Luego calcula $b = a^e \pmod{n}$, y envía este número al usuario $\mathcal{X}$.

- (3) Usando su clave privada, $\mathcal{X}$ calcula $c = b^d \bmod n$. Veamos que esto le permite descifrar el mensaje, es decir, le permite conocer a .

En efecto, supongamos en primer término que a es coprimo con n . Resulta en tal caso (las congruencias son módulo n):

$$c \equiv b^d \equiv (a^e)^d = a^{ed} = a^{t\phi(n)+1} = (a^{\phi(n)})^t a \equiv a,$$

ya que $a^{\phi(n)} \equiv 1$ por el teorema de Fermat–Euler. Puesto que tanto a como c son menores que n concluimos que $c = a$.

Si a no es coprimo con n , supongamos sin pérdida de generalidad que a es divisible por p pero no por q (no puede ser divisible por ambos, ya que $a < n$). Teniendo en cuenta que $q - 1 \mid \phi(n)$, obtenemos entonces las relaciones:

$$\begin{aligned} c &\equiv b^d \equiv a^{ed} \equiv 0 \equiv a \pmod{p} \\ c &\equiv b^d \equiv a^{ed} = a^{t\phi(n)+1} = (a^{\phi(n)})^t a \equiv a \pmod{q}. \end{aligned}$$

Sigue luego —por el teorema chino del resto—, que $c \equiv a \pmod{n}$. En conclusión, $\mathcal{X}$ puede descifrar el mensaje en cualquiera de los dos casos, ya que naturalmente todos los usuarios saben a qué texto corresponde cada número. Más aún, el método que utilicen para hacerlo también puede ser de dominio público.

Observemos que $u \mapsto u^e$ es la función *codificadora* $f_{\mathcal{X}}$ del usuario $\mathcal{X}$, mientras que $u \mapsto u^d$ es la función inversa $f_{\mathcal{X}}^{-1}$ que *decodifica*. O dicho en otras palabras, e es el exponente público para cifrar y d es el exponente secreto para descifrar. Puesto que sólo $\mathcal{X}$ conoce a éste, ningún intruso que interfiera el sistema está en condiciones de descifrar el mensaje.

Vamos a ver con un ejemplo cómo puede transformarse un mensaje —que utiliza el alfabeto de 27 caracteres— en un mensaje numérico. El usuario $\mathcal{X}$ ha elegido los primos 59 y 97 (no se trata por supuesto de una situación real), resultando en tal caso $n = 5723$ y $\phi(n) = 58.96 = 5568$. El exponente de cifrado e de $\mathcal{X}$ será 301, siendo entonces 37 (su inverso módulo 5568) el exponente de descifrado d . Vale decir, (5723, 301) es la clave pública de $\mathcal{X}$, mientras que (59, 97, 37) es la secreta.

Teniendo en cuenta que se trabaja con un alfabeto de 27 caracteres, y que además se verifican las desigualdades $27^2 < n < 27^3$, la idea es separar el texto del mensaje en bloques de dos letras (*digrafos*)¹. Puesto que hemos asignado a cada letra un número entre 0 y 26, puede pensarse que cada bloque $\mathbb{B}$ representa las cifras en base 27 de un número menor que 27^2 ,

¹Si el número de caracteres del mensaje es impar, se le agrega una letra suficientemente infrecuente en el idioma para que no pueda ser interpretada como parte del texto

y por lo tanto menor que n . Si a ese número le aplicamos la función codificadora de $\mathcal{X}$ —utilizando el algoritmo de cálculo de potencias modulares—, obtendremos un número menor que n , y por ende menor que 27^3 . Si ahora tomamos el desarrollo en base 27 de este número, el mismo tendrá 3 cifras (siempre podemos completar a la izquierda con ceros), cada una de las cuales corresponde a una letra del alfabeto. Este bloque de 3 letras (*trigrafo*) será entonces el cifrado del bloque $\mathbb{B}$. Por ejemplo, supongamos que queremos cifrar el digrafo EX . Examinando la tabla de equivalencias, vemos que al mismo le corresponde el par $(4, 24)$, cuyas componentes representan las cifras del número $a = 132 = 4 \cdot 27 + 24$. Aplicando ahora la función de cifrado, obtenemos $b = 2945 = 132^{301} \bmod 5723$.

Una simple cuenta nos muestra que las cifras de b en base 27 son $(4, 1, 2)$, que corresponden a las letras E, B y C , respectivamente. Por lo tanto, el digrafo EX se transforma al cifrarlo en el trigrafo EBC . Por supuesto, para decodificar realizamos el proceso inverso, separando el texto cifrado en bloques de tres caracteres y aplicando a cada bloque (convertido en un número) la función decodificadora del usuario $\mathcal{X}$.

Como ejemplo final, supongamos que $\mathcal{X}$ ha recibido el mensaje cifrado

DBJHJWBPKDXXAMK

La siguiente tabla mostrará en detalle el proceso de descifrado que describimos arriba (las operaciones son módulo 5723):

trigrafo	cifras	b	$a = b^{37}$	cifras	digrafo
DBJ	$(3, 1, 9)$	2223	18	$(0, 18)$	AR
HJW	$(7, 9, 23)$	5369	236	$(8, 20)$	IT
BPK	$(1, 16, 10)$	1171	328	$(12, 4)$	ME
DXX	$(3, 24, 24)$	2859	548	$(20, 8)$	TI
AMK	$(0, 12, 10)$	334	54	$(2, 0)$	CA

Por lo tanto, el texto claro es

ARITMETICA

Ejercicios y problemas propuestos

1. Aplique el método de Fermat para factorizar los números 4601, 8633 y 809009.
2. Resuelva las siguientes ecuaciones:
 - (i) $x^{12} \equiv 1 \pmod{79}$
 - (ii) $x^{35} \equiv -1 \pmod{43}$
 - (iii) $x^9 \equiv -1 \pmod{31}$
 - (iv) $x^8 \equiv -1 \pmod{29}$
 - (v) $x^{28} \equiv -1 \pmod{41}$.
3. ¿Cuál es la probabilidad de que 341 pase el test de Fermat respecto a una base tomada al azar?
4. Verifique que los números 2821, 6601, 172081 y 278545 son pseudoprimos.
5. Sea $m \in \mathbb{N}$ tal que $6m + 1$, $12m + 1$ y $18m + 1$ son primos. Demuestre que $(6m + 1)(12m + 1)(18m + 1)$ es un número de Carmichael. Halle dos ejemplos de esta situación.
6. Aplique algún test de primalidad para decidir cuáles de los números 3071, 7573, 10541, 12553 y 120781 son primos.

7. Mediante la aplicación de una transformación César se ha obtenido el texto cifrado

QDQBYJYJTUVIUSLUDSYQ

Descubra el texto claro.

8. Un usuario del criptosistema RSA —cuya clave privada es $(79, 199, 43)$ — ha recibido el mensaje codificado

MWLHKMSUVUGT

que fue cifrado con el sistema de separación en digrafos y trigrafos que describimos en la última sección de este capítulo. Descifre el mensaje y halle además su clave pública.